

Project 3

zkVM-guest API Standardization

5th Breakout Call · standards recap & new proposals

Standards Recap

Accepted (navy) · In review (amber) · Draft (grey)

1 ACCEPTED

RISC-V Target

Minimal ISA riscv64im_zicclsm. Every conforming zkVM uses this 64-bit target.

3 ACCEPTED

Cryptographic Accelerators

Standard C interface for EVM precompile-equivalent operations.

5 ACCEPTED

Memory Layout Restrictions

Each vendor owns its memory map; portability stays at the source level.

7 ACCEPTED

Instruction Address Misaligned

A misaligned jump target aborts. No rounding, no recovery.

2 ACCEPTED

IO Interface

Portable read_input / write_output for private inputs and public outputs.

4 ACCEPTED

Termination Semantics

Success may produce a proof; failure must not. 0 = success, non-zero = error.

6 ACCEPTED

Memory Safety Guard Regions

Mandatory null-pointer trap and stack guard; violations abort, never corrupt.

8 IN REVIEW

Static Library & Linker Script

The vendor-supplied guest runtime. In comment period — detailed next.

DRAFT — covered next:

ELF Loading & Validation (#33)

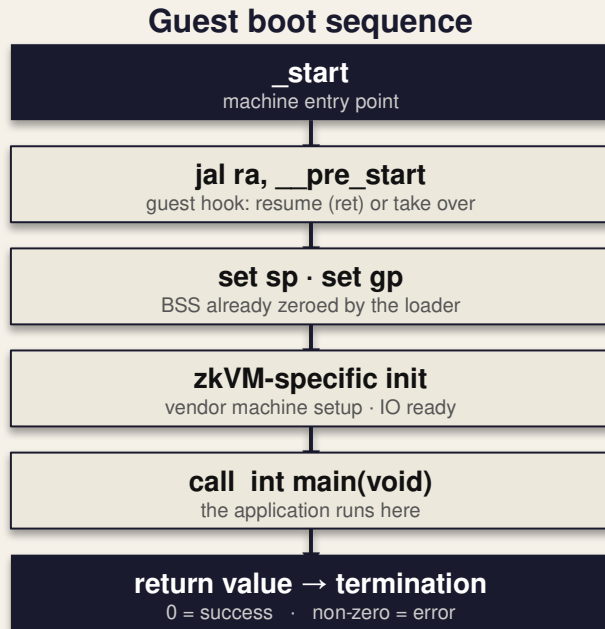
Accelerated Memory Operations (#32)

In Review: Static Library & Linker Script

PR #28 · the runtime each vendor ships with its zkVM

What the standard defines

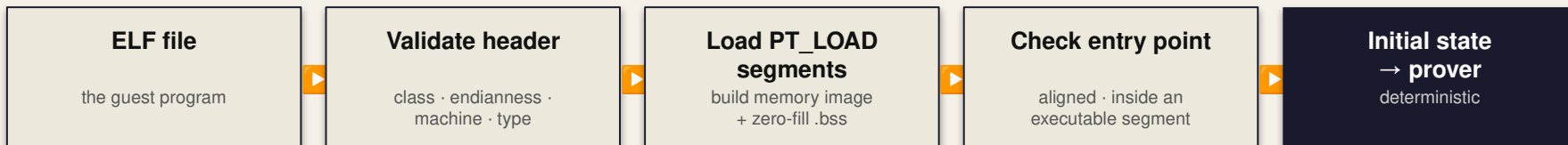
- One static library (.a) holds everything zkVM-specific: `_start`, IO, crypto accelerators, and a `__pre_start` hook.
- A linker script (GNU ld + LLD) sets the entry point, keeps code and data in separate segments, and exposes `_heap_start` / `_heap_end`.
- The guest writes only `int main(void)`; its return value drives termination.



Discussion: one open question — how a guest takes the heap. Resolved in favour of the simple `_heap_start` / `_heap_end` symbols.

New Draft: ELF Loading & Validation

PR #33 · one precise, deterministic way to read a guest ELF into initial state



Section headers are ignored for loading — image comes from program headers.

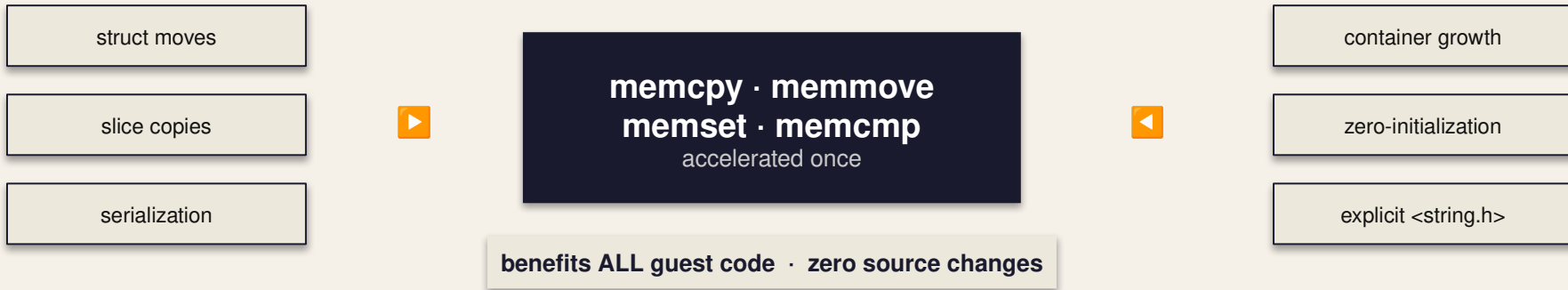
What it standardizes

- The ELF is the authoritative description of the initial machine state.
- The memory image comes from the program headers (the loadable segments) — not section headers, so stripped binaries still load.
- Strict checks: 64-bit, little-endian, RISC-V, executable type; no overlaps; writable-or-executable, never both.
- An invalid ELF is rejected up front — no proof is produced.

Memory addresses and region sizes stay vendor-defined — this standard fixes how an ELF is read, not where memory lives.

New Draft: Accelerated Memory Operations

PR #32 · speed up the memory copies every program already makes



- These functions dominate real Ethereum workloads, and compilers emit them implicitly — so one accelerated symbol speeds up everything.
- It is a symbol contract, not a new API: the interface is plain `<string.h>`.
- Acceleration is optional, but the behaviour must match the standard C functions exactly, for every input.

Two standards are open for your review now: PR #33 and PR #32 — please comment on the PRs.

New Draft: Logging Function

PR #27 · an external contribution from @rubo · extends the IO interface

Part of the proven relation

`read_input()`

`write_output()`

Diagnostic only — not proven

NEW

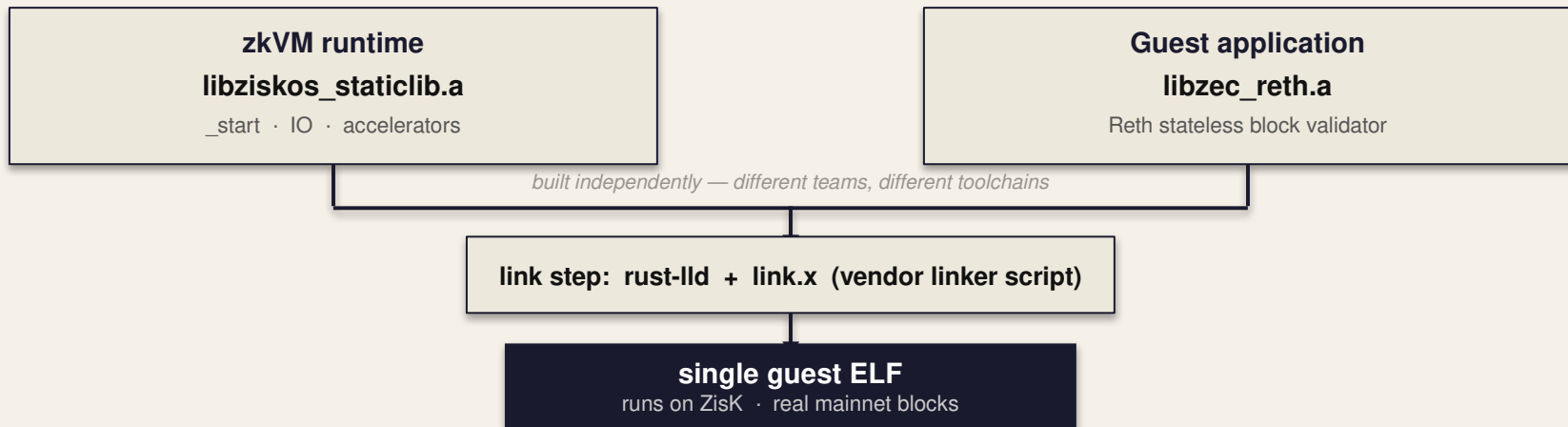
`write_log(utf8_bytes, len)`

UTF-8 text · production provers may ignore it

- Adds one function to the IO interface — `write_log` — for `println`-style developer diagnostics.
- Takes UTF-8 bytes, so it behaves the same way from any language.
- It is not public output and not part of the proven statement, and it must not change execution.

Proof of Concept: Independent Build & Link

zisk-eth-client PR #86 · the static-library model from Breakout 4, working end to end



— The runtime and the application are two separate static libraries — neither needs to know the other's internals.

— A final, independent step links them with the vendor's linker script into one ELF — exactly the model from the standard in review.

Proof of concept: github.com/0xPolygonHermes/zisk-eth-client/pull/86

Something similar was achieved for Zilkworm, on Zisk and SP1, by @Developeruche and Ignacio — github.com/eth-act/zilkworm-stateless